

JAVA EE 8 FRONT-END - ANGULAR (V19)

Módulo 1: Introducción y Setup

Introducción al Módulo

La evolución del ecosistema frontend empresarial demanda arquitecturas de alto rendimiento, bajo acoplamiento y reactividad granular. En este módulo comprenderás los fundamentos y novedades de Angular 19, contrastando los paradigmas clásicos de NgModules frente a los modernos Componentes Standalone (ver ANEXO I). A través del aprovisionamiento con Angular CLI v19 (ver ANEXO II y ANEXO III), el análisis pormenorizado de la estructura limpia de proyecto y el desarrollo de ejercicios interactivos basados en Signals (ver ANEXO I) y Resource API, adquirirás las competencias requeridas para liderar desarrollos de cliente robustos y escalables integrando armónicamente las bases de marcado (ver ANEXO IV), maquetación web (ver ANEXO V) y dinamismo en cliente (ver ANEXO VI).

Estructura Temática y Hoja de Ruta Formativa

Este manual técnico está concebido como una guía teórico-práctica secuencial articulada en los siguientes puntos principales:

- **0 Prólogo: Evolución de las Arquitecturas Web:** Trayectoria histórica desde la web monolítica tradicional con recarga total en servidor hasta la reactividad contemporánea con SPAs.
- **1 ¿Qué es Angular y qué hay de nuevo en Angular 19?:** Fundamentos de la plataforma, reactividad granular y modelo de rendimiento $\mathcal{O}(k)$ frente a Zone.js.
- **2 Standalone Components vs Módulos Tradicionales (NgModule):** Arquitectura sin módulos, optimización de empaquetado (*Tree-Shaking*) y reducción de boilerplate.
- **3 Instalación de Angular CLI y Creación de Proyecto:** Prerrequisitos de entorno, instalación global y aprovisionamiento con banderas autónomas.
- **4 Estructura Base de un Proyecto Angular 19:** Desglose del árbol de archivos, bootstrap directo, proveedores globales y ejercicios prácticos de aplicación (Signals y Resource API).
- **5 Ejercicio Final:** Caso de integración empresarial que implementa un panel de control distribuido mediante reactividad con `linkedSignal`.
- **6 Resumen Ejecutivo del Conocimiento Adquirido:** Síntesis formal de las capacidades, patrones y directrices asimiladas en la unidad.

Anexos Técnicos de Referencia:

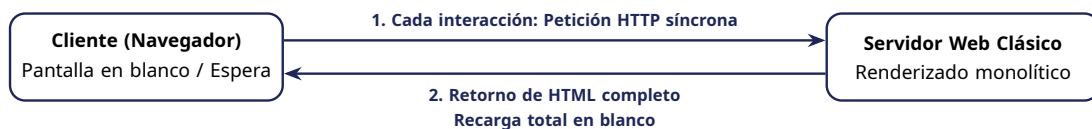
- **ANEXO I (Glosario de Términos):** Compendio conceptual con la definición de términos técnicos citados con (ver ANEXO I).
- **ANEXO II (Guía de Instalación):** Guía paso a paso y matriz de compatibilidad de software para el aprovisionamiento, citada con (ver ANEXO II).
- **ANEXO III (Manual Técnico Avanzado de Angular CLI):** Automatización, Schematics, scaffolding, guards funcionales, directivas y comandos de calidad, citado con (ver ANEXO III).
- **ANEXO IV (Guía Básica de HTML):** Fundamentos del desarrollo estructurado, anatomía de etiquetas, plantilla universal HTML5 y marcado semántico, citado con (ver ANEXO IV).

- **ANEXO V (Guía Básica de CSS):** Fundamentos de maquetación, sintaxis de reglas, métodos de integración, selectores, modelo de caja y propiedades esenciales, citado con **(ver ANEXO V)**.
- **ANEXO VI (Guía Básica de JavaScript):** Fundamentos de dinamismo en cliente, sintaxis ECMAScript moderna, manipulación del DOM, eventos y asincronía, citado con **(ver ANEXO VI)**.

0 Prólogo: Evolución de las Arquitecturas Web

Para comprender el origen y la necesidad de los frameworks reactivos modernos como Angular 19, resulta indispensable analizar la evolución del diálogo entre el cliente (el navegador) y el servidor a lo largo de las últimas tres décadas.

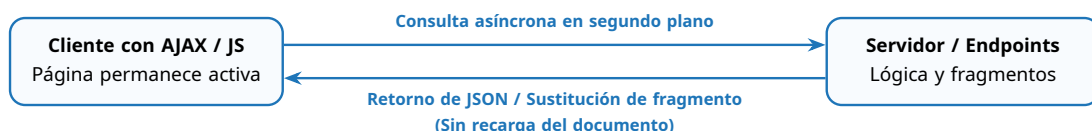
1. La Web Tradicional y el Renderizado Monolítico en Servidor



Mecánica operativa: En los albores de la web, todo el cómputo y la generación visual residían exclusivamente en el servidor (PHP, JSP, ASP tradicional). Cada vez que el usuario hacía clic en un enlace o enviaba un formulario, el navegador emitía una petición HTTP síncrona completa.

El problema de la experiencia: El navegador desechaba por completo el documento existente en memoria, provocando una pantalla en blanco (*white flash*) mientras aguardaba la respuesta. El servidor reconstruía la página íntegra —incluyendo cabeceras, menús y estilos idénticos—, lo que generaba un tráfico de red redundante y una sensación constante de lentitud e interrupción.

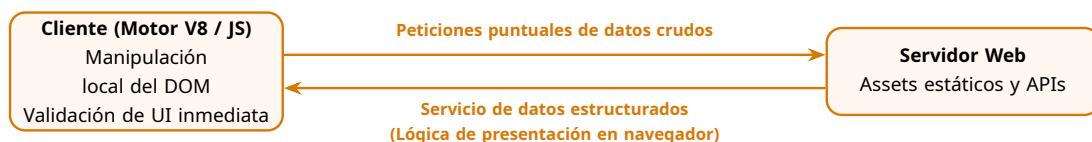
2. La Revolución de AJAX y la Comunicación Asíncrona



El salto cualitativo: Con la estandarización del objeto XMLHttpRequest (y el posterior auge de jQuery), nació el concepto de AJAX (*Asynchronous JavaScript and XML*). Por primera vez, el navegador adquirió la capacidad de realizar consultas silenciosas al servidor en segundo plano sin congelar la ventana activa.

Impacto práctico: Al recibir datos en formato XML o JSON, un script en cliente actualizaba únicamente una sección específica del DOM (por ejemplo, el resultado de una búsqueda o el contador de un carrito). La página principal permanecía intacta, reduciendo notablemente los tiempos de espera y el ancho de banda consumido.

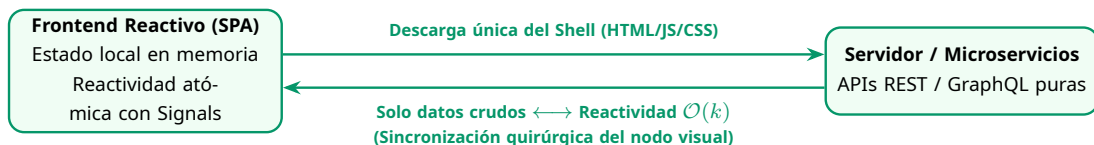
3. Dinamismo Enriquecido en Cliente con JavaScript Moderno



Mayor peso en el navegador: Conforme los navegadores incorporaron motores de ejecución ultra-rápidos (como V8), JavaScript dejó de ser un simple complemento cosmético (ver ANEXO VI). Tareas como la validación instantánea de credenciales, menús interactivos, transiciones y pestañas pasaron a resolverse directamente en la máquina del usuario sin tocar la red.

El reto del mantenimiento: A medida que la interfaz ganaba interactividad, la manipulación directa y manual del árbol DOM con código imperativo se volvió compleja e insostenible (*spaghetti code*), haciendo patente la necesidad de herramientas estructuradas con patrones de diseño rigurosos.

4. La Era de los Frameworks Reactivos (SPAs) y Signals en Angular 19



Arquitectura Single Page Application: Las interfaces empresariales contemporáneas se conciben como aplicaciones de escritorio que residen en la nube (ver ANEXO I). El navegador descarga el contenedor estructural (*shell*) una sola vez. A partir de ahí, la navegación es instantánea: el enrutador gestiona las vistas en memoria y el servidor se limita a proveer datos estructurados vía APIs REST o GraphQL.

La reactividad atómica de Angular 19: Gracias a las primitivas reactivas como los *Signals* (ver ANEXO I), el framework ya no necesita recorrer recursivamente todo el árbol de componentes para detectar cambios. La sincronización se produce con coste constante $\mathcal{O}(k)$, actualizando de manera quirúrgica exclusivamente el nodo visual que depende del dato modificado.

1 ¿Qué es Angular y qué hay de nuevo en Angular 19?

Angular: Es una plataforma de desarrollo y un framework de código abierto basado en TypeScript, liderado por Google, diseñado específicamente para construir aplicaciones web de nivel empresarial y de página única (*Single Page Applications* - SPAs, ver ANEXO I). A diferencia de otras librerías de interfaz de usuario, Angular se distingue por ser un framework “opinado” (*opinionated*), proporcionando una arquitectura estricta y soluciones preconfiguradas para el enrutamiento, la gestión de formularios, la comunicación HTTP y la inyección de dependencias (ver ANEXO I). Esto minimiza la fragmentación técnica y garantiza la escalabilidad en proyectos de gran envergadura.

Evolución hacia Angular 19: Con el lanzamiento de Angular 19, el framework consolida su paradigma de desarrollo moderno (iniciado de manera disruptiva en la versión 14 con los componentes autónomos y perfeccionado con los Signals, ver ANEXO I). El objetivo fundamental de esta versión es la optimización de la reactividad fina, la simplificación extrema del código estándar (*boilerplate*) y la entrega de un rendimiento en el servidor y cliente excepcionalmente eficiente.

Modelado del Rendimiento en Angular 19

La introducción de la reactividad basada en Signals (ver ANEXO I) elimina la necesidad de recorrer recursivamente todo el árbol de componentes mediante Zone.js (ver ANEXO I). El costo temporal de actualización de la vista se reduce significativamente:

Modelo Clásico (Zone.js)

$$T_{update} = \mathcal{O}(N)$$

Donde N es el número total de componentes en el árbol jerárquico de la aplicación y k representa únicamente la cantidad de nodos consumidores que dependen directamente de la señal modificada ($k \ll N$).

Signals (Angular 19)

$$T_{update} = \mathcal{O}(k)$$

Novedades Principales en Angular 19:

- **Estabilización Completa de Signals (ver ANEXO I):** Los mecanismos de reactividad pura basada en señales se convierten en el estándar recomendado de producción. Conceptos como `signal()`, `computed()` y `effect()` están optimizados para evitar fugas de memoria y sincronizaciones fallidas.
- **Nueva API `linkedSignal` (ver ANEXO I):** Angular 19 introduce `linkedSignal`, una herramienta avanzada diseñada para mantener una señal mutable sincronizada de forma reactiva con un valor de origen computable, superando los problemas de rendimiento del uso directo de efectos mutadores.
- **Soporte Nativo de `Resource` y `rxResource`:** Simplificación de la carga de datos asíncronos mediante señales. Permite enlazar directamente llamadas HTTP a flujos de datos basados en estados de señales (`loading`, `resolved`, `error`) sin suscripciones manuales de RxJS (ver ANEXO I).
- **Server-Side Rendering (SSR, ver ANEXO I) Híbrido Avanzado:** El motor de renderizado del servidor en Angular 19 reduce los tiempos de hidratación activa (ver ANEXO I) gracias a la serialización mejorada de las transferencias de estado y la hidratación incremental.
- **Soporte TypeScript 5.5+:** Aprovechamiento de las últimas innovaciones del lenguaje como los predicados de tipos inferidos (*inferred type predicates*), mejorando sustancialmente el análisis estático de código.

2 Standalone Components vs Módulos Tradicionales (NgModule)

Históricamente, Angular estructuraba el acoplamiento y la compilación de sus elementos mediante unidades lógicas denominadas *NgModules* (representados por la clase decorada con `@NgModule`). Un módulo actuaba como un delimitador de contexto de compilación que agrupaba componentes, directivas, pipes y servicios.

La Revolución de los Componentes Autónomos (Standalone, ver ANEXO I): Introducidos para simplificar el modelo mental del desarrollador, los componentes autónomos prescinden completamente de los archivos de módulo. En su lugar, el propio componente declara directamente sus dependencias de importación en sus metadatos de decorador.

Cuadro 1: Comparación técnica entre NgModules y Componentes Standalone

Criterio Técnico	Tradicionales Módulos (@NgModule)	Componentes Standalone (Angular 19)
Declaración de Dependencias	Declarado en el arreglo de <code>declarations</code> de un <code>NgModule</code> común.	Declarado directamente dentro del arreglo <code>imports</code> del propio decorador.
Carga Perezosa (Lazy Loading)	Requiere importar el archivo de módulo mediante <code>loadChildren</code> apuntando al módulo.	Importación directa del componente mediante <code>loadComponent</code> . Sencillo y directo.
Efectividad del Tree-Shaking (ver ANEXO I)	Complejo. A menudo se arrastran elementos inutilizados declarados dentro del mismo módulo.	Óptimo. El compilador sabe exactamente qué componente requiere qué directiva a nivel unitario.
Complejidad Cognitiva	Alta. Requiere mantener múltiples archivos sincronizados para una sola interfaz de usuario.	Baja. Acoplamiento directo de la lógica, diseño y dependencias en un solo archivo base.

Definición de Componente Standalone en Angular 19:

```
// app.component.ts - Componente Autonomo
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { MatButtonModule } from '@angular/material/button';

@Component({
  selector: 'app-root',
  standalone: true, // Definición explícita (por defecto en Angular 19)
  imports: [CommonModule, MatButtonModule], // Dependencias directas
  template: `
    <main class="p-4">
      <h1>Bienvenido a Angular 19</h1>
      <button mat-raised-button color="primary">Acción Autónoma</button>
    </main>
  `,
  styles: [
    h1 { font-family: 'Tahoma', sans-serif; color: #1F2758; }
  ]
})
export class AppComponent {}
```

Listing 1: p2-standalone/app.component.ts

3 Instalación de Angular CLI y Creación de Proyecto con `--standalone`

El *Angular Command Line Interface* (CLI, ver ANEXO I, ANEXO II y ANEXO III) es el pilar central para la automatización de procesos de andamiaje, pruebas, construcción y despliegue dentro del ecosistema de Angular.

Requisitos Previos de Entorno (ver ANEXO II): Antes de proceder, es indispensable asegurar un entorno de ejecución JavaScript adecuado. Angular 19 requiere de manera mandatoria versiones LTS activas de Node.js y un gestor de paquetes NPM compatible.

Verificación de Prerrequisitos en Terminal (ver ANEXO II):

```
node --version # Debe retornar >= v18.19.1 o >= v20.x, v22.x
npm --version  # Generalmente administrado junto a Node.js
```

Paso 1: Instalación Global de Angular CLI v19 (ver ANEXO II y ANEXO III): Ejecuta el siguiente comando para aprovisionar el CLI globalmente en tu sistema operativo:

```
npm install -g @angular/cli@19
```

Paso 2: Generación del Nuevo Proyecto Autónomo (ver ANEXO II y ANEXO III): Con el CLI instalado, iniciamos la construcción de la estructura del proyecto. A partir de Angular 17/18 y de manera consolidada en la versión 19, el CLI crea proyectos con arquitectura Standalone (ver ANEXO I) por defecto. El comando de creación se estructura de la siguiente forma:

```
ng new angular-nineteen-mastery --standalone=true --routing=true --style=scss
```

Desglose de Parámetros Utilizados:

- `--standalone=true`: Configura la aplicación para prescindir de archivos `app.module.ts` y arrancar directamente el componente raíz mediante bootstrap autónomo.
- `--routing=true`: Inicializa el sistema de enrutamiento integrado de Angular, creando el archivo de configuración correspondiente.
- `--style=scss`: Define el preprocesador Sass para la gestión avanzada y anidada de hojas de estilo a nivel de componentes (ver ANEXO V).

Paso 3: Ejecución en Servidor de Desarrollo local (ver ANEXO II y ANEXO III): Accede al directorio creado y levanta la infraestructura local de compilación incremental en memoria:

```
cd angular-nineteen-mastery
ng serve --open
```

El flag `--open` levantará de forma automática tu navegador web predeterminado apuntando a la dirección local `http://localhost:4200`.

4 Estructura Base de un Proyecto Angular 19

Un proyecto generado en Angular 19 posee una arquitectura notablemente limpia debido a la remoción de los NgModules y la optimización de los ficheros de configuración global. A continuación, se presenta la representación del árbol de directorios utilizando una estructura ASCII segura que previene fallos de caracteres de fuente, seguida de la explicación de cada elemento clave:

```
angular-nineteen-mastery/
|-- .editorconfig
|-- angular.json           <-- Metadatos y pipeline CLI
|-- package.json          <-- Dependencias y scripts de ejecucion
|-- tsconfig.json         <-- Configuracion TypeScript
`-- src/
    |-- main.ts           <-- Punto de entrada e inicializacion
    |-- index.html        <-- Plantilla HTML contenedora global
    `-- app/
        |-- app.config.ts  <-- Proveedores globales (Rutas, Hydration)
        |-- app.routes.ts  <-- Definicion de mapeos de enrutamiento
        |-- app.component.ts <-- Componente autonomo raiz
        |-- app.component.html <-- Vista HTML del componente raiz
        `-- app.component.scss <-- Estilo SCSS del componente raiz
```

Ficheros de Configuración Críticos

1. src/main.ts: En una arquitectura standalone (ver ANEXO I), ya no se inicializa un módulo raíz (`platformBrowserDynamic().bootstrapModule(...)`). En su lugar, el arranque se efectúa directamente sobre el componente raíz mediante la función optimizada `bootstrapApplication()`:

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

Listing 2: p4-config/src/main.ts

2. src/app/app.config.ts: Centraliza la inyección de dependencias global utilizando una arquitectura orientada a proveedores (*Providers*). Aquí se configuran servicios clave como el enrutador, la hidratación del lado del servidor (ver ANEXO I) y la detección de cambios:

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';
import { provideClientHydration } from '@angular/platform-browser';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }), // Agrupacion de eventos de ZoneJS
    provideRouter(routes),                               // Inyeccion de rutas de navegacion
    provideClientHydration()                             // Soporte de hidratacion para SSR
  ]
};
```

Listing 3: p4-config/src/app/app.config.ts

3. angular.json: El manifiesto del proyecto. Define los entornos de compilación (*development*, *production*), los assets estáticos, scripts externos y los puntos de entrada para la compilación del framework (ver ANEXO III).

Práctica Guiada: Ejercicios de Implementación en la Estructura Base

A continuación se desarrollan dos casos prácticos secuenciales que consolidan la manipulación de componentes autónomos, reactividad granular y gestión asíncrona dentro de la arquitectura base del proyecto recién estructurado.

Ejercicio 1: Creación de Componente de Monitoreo de Rendimiento usando Signals

Enunciado

En este ejercicio modelaremos de forma interactiva una visualización del rendimiento de procesamiento en Angular 19. Usaremos señales (ver ANEXO I) para almacenar el retardo de latencia en milisegundos y calcularemos la eficiencia derivada mediante una señal computada dependiente. La vista debe permitir modificar la latencia y reflejar inmediatamente los cambios de diagnóstico en tiempo real.

Solución

```
import { Component, signal, computed } from '@angular/core';
```

```

import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-perf-monitor',
  standalone: true,
  imports: [CommonModule],
  template: `
    <div class="border border-slate-200 p-6 rounded-lg bg-slate-50">
      <h3 class="text-lg font-bold text-[#1F2758]">Monitor de Rendimiento Angular 19</h3>
      <p class="mt-2 text-sm">Latencia Actual: <strong>{{ latencia() }} ms</strong></p>
      <p class="text-sm">Estado de Eficiencia: <strong>{{ eficiencia() }}</strong></p>
      <div class="mt-4 flex gap-2">
        <button (click)="incrementarLatencia()" class="bg-[#1F2758] text-white px-3 py-1.5
        ↪ rounded text-xs">
          Forzar Sobrecarga (+50ms)
        </button>
        <button (click)="restablecerLatencia()" class="bg-emerald-600 text-white px-3 py-1.5
        ↪ rounded text-xs">
          Optimizar Red
        </button>
      </div>
    </div>
  `
})
export class PerfMonitorComponent {
  latencia = signal<number>(20);

  eficiencia = computed(() => {
    const lat = this.latencia();
    if (lat < 50) return 'Excelente (Optima reactividad)';
    if (lat >= 50 && lat < 150) return 'Regular (Zona ZoneJS)';
    return 'Peligro (Sobrecarga de renderizado)';
  });

  incrementarLatencia() {
    this.latencia.update(current => current + 50);
  }

  restablecerLatencia() {
    this.latencia.set(20);
  }
}

```

Listing 4: p4-ej1/perf-monitor.component.ts

Explicación

- **Definición y mutabilidad de la señal (signal(20)):** Reserva un espacio reactivo que mantiene el valor primario en memoria (ver ANEXO I). La mutación se efectúa mediante los métodos explícitos `.set()` o `.update()`, garantizando trazabilidad y control de cambios.
- **Reactividad pura dependiente (computed()):** Define una lectura reactiva pura que registra la señal `latencia()` como su dependencia interna. Si `latencia()` cambia, la señal computada se invalida y recalcula en el siguiente ciclo de render de manera eficiente, optimizando sustancialmente el consumo de procesador frente al modelo clásico de Zone.js (ver ANEXO I).

Ejercicio 2: Flujo asíncrono con Resource API en Angular 19

Enunciado

En Angular 19, la API `resource` simplifica la carga interactiva de recursos externos en sintonía con las señales (ver ANEXO I), evitando el uso recursivo de pipes asíncronos clásicos de RxJS (ver ANEXO I). Desarrolla un componente autónomo que reciba un identificador numérico de usuario y realice la recuperación asíncrona de sus datos, exponiendo de forma transparente los estados de carga (`isLoading`), error y resolución.

Solución

```
import { Component, resource, signal } from '@angular/core';
import { CommonModule } from '@angular/common';

interface User {
  id: number;
  name: string;
}

@Component({
  selector: 'app-user-loader',
  standalone: true,
  imports: [CommonModule],
  template: `
    <div class="p-6 bg-white border border-slate-200 rounded-lg shadow-sm">
      <h3 class="text-lg font-bold text-[#1F2758]">Buscador de Usuarios Reactivo</h3>
      <div class="mt-3">
        <label class="block text-xs font-semibold text-slate-600">ID de Usuario:</label>
        <input
          type="number"
          [value]="userId()"
          (input)="actualizarId($event)"
          class="border border-slate-300 rounded px-2 py-1 text-sm mt-1" />
      </div>
      <div class="mt-4">
        @if (userResource.isLoading()) {
          <p class="text-blue-500 text-sm animate-pulse">Cargando informacion del usuario...</p>
        } @else if (userResource.error()) {
          <p class="text-red-500 text-sm">Error al procesar la peticion.</p>
        } @else if (userResource.value()) {
          <div class="bg-emerald-50 border border-emerald-200 p-3 rounded">
            <p class="text-sm font-bold text-emerald-800">Usuario Encontrado:</p>
            <p class="text-xs text-emerald-900">Nombre: {{ userResource.value()?.name }}</p>
          </div>
        }
      </div>
    </div>
  `
})
export class UserLoaderComponent {
  userId = signal<number>(1);

  userResource = resource({
    request: () => ({ id: this.userId() }),
  })
}
```

```
loader: async ({ request }) => {
  const url = `https://jsonplaceholder.typicode.com/users/${request.id}`;
  const response = await fetch(url);
  if (!response.ok) throw new Error('Usuario inexistente');
  return await response.json() as User;
}

actualizarId(event: Event) {
  const input = event.target as HTMLInputElement;
  const value = parseInt(input.value, 10);
  if (!isNaN(value)) {
    this.userId.set(value);
  }
}
}
```

Listing 5: p4-ej2/user-loader.component.ts

Explicación

- **Carga declarativa sincronizada con señales:** Al vincular la propiedad `request` del recurso a `this.userId()`, cualquier invocación a `this.userId.set()` reinicia automáticamente la ejecución asíncrona de la función `loader`.
- **Estados derivados unificados:** La Resource API proporciona señales intrínsecas como `.isLoading()`, `.error()` y `.value()`, permitiendo al nuevo bloque de control `@if` del template reaccionar de forma limpia sin suscripciones manuales de RxJS (ver ANEXO I).

5 Ejercicio Final

Panel de Control de Microservicios Distribuidos

Enunciado

Has sido contratado para estructurar el panel de control inicial de microservicios en una organización de computación de alto rendimiento. Deberás crear de manera autónoma en Angular 19 un panel que:

1. Muestre un estado general dependiente de la agregación de latencias de múltiples microservicios (empleando una señal computada, ver ANEXO I).
2. Sincronice automáticamente la categoría de la prueba usando `linkedSignal` de Angular 19 (ver ANEXO I) de acuerdo al servidor seleccionado.
3. Posea una interfaz interactiva estilizada utilizando los lineamientos corporativos (ver ANEXO V).

Solución

```
import { Component, signal, computed, linkedSignal } from '@angular/core';
```

```

import { CommonModule } from '@angular/common';

interface Microservicio {
  nombre: string;
  latencia: number;
}

@Component({
  selector: 'app-dashboard-integrador',
  standalone: true,
  imports: [CommonModule],
  template: `
    <div class="p-8 border-2 border-[#1F2758] rounded-xl bg-slate-50">
      <div class="flex justify-between items-center border-b border-slate-200 pb-4 mb-6">
        <h2 class="text-xl font-bold text-[#1F2758]">PANEL DE CONTROL DE MICROSERVICIOS</h2>
        <span class="px-3 py-1 bg-[#1F2758] text-white text-xs font-semibold rounded-full">
          Angular v19.x Stable
        </span>
      </div>

      <div class="mb-6 bg-white p-4 rounded-lg border border-slate-200">
        <p class="text-sm font-bold text-slate-700 mb-2">1. Seleccionar Servidor de Trafico:</p>
        <div class="flex gap-4">
          <button (click)="seleccionarServidor('Norteamerica')"
            [class]="servidorActivo() === 'Norteamerica'
              ? 'bg-[#1F2758] text-white px-3 py-2 rounded text-xs'
              : 'bg-slate-200 text-slate-700 px-3 py-2 rounded text-xs'">
            Nodo America del Norte
          </button>
          <button (click)="seleccionarServidor('Europa')"
            [class]="servidorActivo() === 'Europa'
              ? 'bg-[#1F2758] text-white px-3 py-2 rounded text-xs'
              : 'bg-slate-200 text-slate-700 px-3 py-2 rounded text-xs'">
            Nodo Europa Occidental
          </button>
        </div>

        <div class="mt-4 p-3 bg-amber-50 border-l-4 border-amber-500 rounded-r">
          <p class="text-xs text-amber-800">
            <strong>Variable Vinculada (linkedSignal):</strong> La ruta por defecto se actualiza
            automaticamente cuando cambia el nodo geografico, permitiendo edicion local.
          </p>
          <p class="text-sm mt-1 font-mono text-amber-900">
            Ruta de Pruebas: <strong>{{ rutaServidor() }}</strong>
          </p>
          <button (click)="personalizarRuta()" class="mt-2 text-xs text-[#1F2758] underline font
            ↪ -semibold">
            Personalizar Ruta Manualmente
          </button>
        </div>
      </div>

      <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div class="bg-white p-4 rounded-lg border border-slate-200">
          <h3 class="text-sm font-bold text-[#1F2758] mb-3">2. Monitoreo de Latencias
          ↪ Individuales</h3>

```

```

    <div class="space-y-3">
      @for (ms of microservicios(); track ms.nombre) {
        <div class="flex justify-between items-center bg-slate-50 p-2 rounded">
          <span class="text-xs font-mono font-bold">{{ ms.nombre }}</span>
          <div class="flex items-center gap-3">
            <span class="text-xs font-bold" [class]="ms.latencia > 120 ? 'text-red-500' :
↪ 'text-emerald-600'">
              {{ ms.latencia }} ms
            </span>
            <button (click)="simularRetardo(ms.nombre)" class="text-[10px] bg-slate-200 px
↪ -2 py-0.5 rounded">
              +50ms
            </button>
          </div>
        </div>
      }
    </div>
    <button (click)="reiniciarMetricas()" class="mt-4 w-full bg-slate-800 text-white text-
↪ xs py-2 rounded">
      Restaurar Metricas por Defecto
    </button>
  </div>

  <div class="bg-white p-4 rounded-lg border border-slate-200 flex flex-col justify-
↪ between">
    <div>
      <h3 class="text-sm font-bold text-[#1F2758] mb-3">3. Diagnostico Matematico
↪ Computado</h3>
      <p class="text-xs text-slate-600 leading-relaxed mb-3">
        Calculamos la latencia ponderada promedio ( $\mu_L$ ) para estimar la salud integral
↪ :
      </p>
      <div class="bg-slate-50 p-3 rounded text-center my-2">
        
$$\mu_L = \frac{1}{M} \sum_{i=1}^M L_i = \mathbf{\{ \{ latenciaPromedio() \} \}}$$

↪ { ms}
      </div>
    </div>
    <div class="mt-4 p-4 rounded text-center"
      [class]="estadoSalud().saludable ? 'bg-emerald-100 text-emerald-800' : 'bg-red-100
↪ text-red-800'">
      <span class="text-xs uppercase tracking-wider font-bold">Diagnostico Final:</span>
      <p class="text-lg font-bold">{{ estadoSalud().mensaje }}</p>
    </div>
  </div>
</div>
})
export class DashboardIntegradorComponent {
  microservicios = signal<Microservicio[]>([
    { nombre: 'API Gateway', latencia: 45 },
    { nombre: 'Autenticacion', latencia: 80 },
    { nombre: 'Procesamiento Pagos', latencia: 150 },
    { nombre: 'Catalogo Productos', latencia: 60 }
  ]);
}

```

```
servidorActivo = signal<string>('Norteamerica');

rutaServidor = linkedSignal({
  source: this.servidorActivo,
  computation: (nuevoServidor) => {
    return nuevoServidor === 'Norteamerica'
      ? 'https://us-api.servicios-cloud.org/v1'
      : 'https://eu-api.servicios-cloud.org/v1';
  }
});

latenciaPromedio = computed(() => {
  const lista = this.microservicios();
  if (lista.length === 0) return 0;
  const suma = lista.reduce((total, ms) => total + ms.latencia, 0);
  return Math.round(suma / lista.length);
});

estadoSalud = computed(() => {
  const promedio = this.latenciaPromedio();
  if (promedio < 90) {
    return { saludable: true, mensaje: 'Estable (SLA Garantizado)' };
  } else {
    return { saludable: false, mensaje: 'Alerta de Retardo (Fallo SLA)' };
  }
});

seleccionarServidor(servidor: string) {
  this.servidorActivo.set(servidor);
}

personalizarRuta() {
  this.rutaServidor.set('https://custom-gateway.local-dev.net/dev');
}

simularRetardo(nombre: string) {
  this.microservicios.update(lista =>
    lista.map(ms => ms.nombre === nombre ? { ...ms, latencia: ms.latencia + 50 } : ms)
  );
}

reiniciarMetricas() {
  this.microservicios.set([
    { nombre: 'API Gateway', latencia: 45 },
    { nombre: 'Autenticacion', latencia: 80 },
    { nombre: 'Procesamiento Pagos', latencia: 150 },
    { nombre: 'Catalogo Productos', latencia: 60 }
  ]);
}
```

Listing 6: p5-ejfinal/dashboard-integrador.component.ts

Explicación

- **Gestión de estado mutable sincronizado con linkedSignal (ver ANEXO I):** Permite que la va-

riable `rutaServidor` se restablezca a su valor predeterminado cada vez que la señal origen `servidorActivo` muta, permitiendo al mismo tiempo modificaciones manuales directas sin romper el ciclo reactivo.

- **Evaluación matemática continua:** La función `computed()` recalcula automáticamente la media aritmética μ_L de latencia en cada alteración de microservicio y suministra el estado derivado de cumplimiento de acuerdos de nivel de servicio (SLA).

6 Resumen Ejecutivo del Conocimiento Adquirido

La consolidación técnica de Angular 19 representa una madurez sin precedentes en la eficiencia operacional del ecosistema frontend de Google. Al finalizar este módulo teórico-práctico, has interiorizado los siguientes principios fundamentales:

- **La reactividad basada en Signals supera sustancialmente el modelo clásico de Zone.js (ver ANEXO I)**, permitiendo actualizar únicamente los componentes modificados con una complejidad constante $\mathcal{O}(k)$.
- **Los Componentes Standalone son ahora el estándar nativo por defecto (ver ANEXO I)**, lo que se traduce en mayor mantenibilidad, mejor encapsulación de código y reducción de código repetitivo de compilación.
- **El uso de APIs novedosas como `linkedSignal` de Angular 19 simplifica los flujos de sincronización condicional (ver ANEXO I)**, eliminando la necesidad de efectos colaterales de actualización manual que degradan el rendimiento global.
- **La creación y aprovisionamiento ágil de proyectos mediante Angular CLI 19 (ver ANEXO II y ANEXO III) sienta las bases de un entorno listo para la producción** y la optimización extrema del renderizado en servidores y clientes (SSR y CSR híbridos, ver ANEXO I).
- **La integración armónica de la estructura HTML semántica (ver ANEXO IV), la arquitectura CSS basada en el Box Model (ver ANEXO V) y el dinamismo con JavaScript moderno (ver ANEXO VI)** garantiza una base robusta, accesible y estilizada para los componentes gráficos de vanguardia.

ANEXO I: Glosario de Términos Técnicos

1. **Single Page Application (SPA):** Aplicación web que carga una única página HTML y actualiza dinámicamente el contenido web mediante interacción asíncrona con servicios de datos, evitando recargas totales de página.
2. **Standalone Component:** Componente autónomo en Angular que no requiere la intermediación de un `NgModule`, administrando de forma interna y explícita el conjunto de dependencias que requiere.
3. **Signals:** Primitivas de reactividad de precisión granular que encapsulan un valor reactivo, notificando de manera directa e inequívoca a sus consumidores sobre cualquier cambio de estado.
4. **Tree-Shaking:** Proceso de optimización de compilación orientado a eliminar código muerto o no utilizado en el empaquetado final (*bundle*), optimizando el tamaño de transferencia.
5. **Zone.js:** Librería encargada de interceptar operaciones asíncronas para disparar la detección de cambios de manera global en versiones clásicas de Angular.
6. **Hydration:** Proceso mediante el cual Angular restaura el estado y la interactividad reactiva en el navegador sobre un documento HTML previamente pre-renderizado en el servidor (SSR).
7. **LinkedSignal:** Estructura avanzada de señal introducida en la v19 que asocia una señal editable a otra señal origen, reiniciando su estado interno cada vez que la señal de origen se actualiza.
8. **Angular CLI:** Interfaz de comandos para la gestión de ciclo de vida completo del desarrollo de software en Angular.
9. **Injection Context:** Contexto especial de ejecución (como el constructor de una clase o la función `inject()`) donde está garantizado el acceso al inyector de dependencias de Angular.
10. **Client-Side Rendering (CSR):** Estrategia donde el navegador descarga un bundle de JavaScript vacío y renderiza toda la interfaz en tiempo de ejecución del lado del cliente.
11. **Server-Side Rendering (SSR):** Proceso en el cual el servidor genera la estructura HTML completa por cada petición, acelerando la primera pintura de contenido (*First Contentful Paint*).
12. **RxJS:** Librería de programación reactiva basada en Observables de flujo secuencial, usada para el manejo de streams de datos altamente asíncronos en Angular.

ANEXO II: Guía de Instalación y Aprovisionamiento

Este anexo consolida de manera estructurada y secuencial todas las operaciones de instalación, verificación de entorno y arranque del ecosistema Angular 19 descritas a lo largo del manual técnico.

1 Matriz de Compatibilidad de Software Base

Herramienta	Versión Requerida	Finalidad Operativa
Node.js	$\geq 18.19.1$ ó $\geq 20.x, 22.x$ (LTS)	Entorno de ejecución JavaScript del lado del servidor.
NPM	$\geq 9.x / 10.x$	Gestor de paquetes oficial para dependencias de Node.js.
Angular CLI	v19.x Stable	Generación de proyectos, servidor local y empaquetado.
TypeScript	5.5+	Compilación tipada estática sobre código fuente ECMAScript.

2 Verificación de Prerrequisitos de Entorno

Antes de iniciar cualquier instalación de paquetes en tu estación de trabajo, abre un terminal y confirma que las herramientas base se encuentran en el PATH del sistema:

```
# 1. Comprobar version activa de Node.js
node --version

# 2. Comprobar version instalada de NPM
npm --version
```

3 Instalación Global de Angular CLI v19

El paquete oficial de herramientas CLI debe instalarse con visibilidad global para disponer de la utilidad `ng` en cualquier ruta del sistema operativo:

```
# Instalacion con alcance global en el sistema
npm install -g @angular/cli@19

# Comprobacion de instalacion y telemetria de Angular CLI
ng version
```

4 Generación y Creación de Proyecto Standalone

Para crear un nuevo proyecto configurado de acuerdo al estándar moderno de Angular 19 (prescindiendo de NgModules), ejecuta:

```
ng new angular-nineteen-mastery --standalone=true --routing=true --style=scss
```

Opciones aplicadas:

- `--standalone=true`: Inicia la arquitectura limpia basada en componentes y bootstrap autónomo.
- `--routing=true`: Aprovisiona el enrutador (`app.routes.ts`) y la configuración global (`app.config.ts`).

- `--style=scss`: Configura la compilación de hojas de estilo mediante Sass/SCSS.

5 Inicialización del Servidor de Desarrollo Local

Una vez generado el andamiaje del proyecto, entra en la carpeta creada y lanza el servidor local con recarga en caliente (*Hot-Module Replacement*):

```
cd angular-nineteen-mastery  
ng serve --open
```

La consola compilará los módulos en memoria y abrirá el navegador en `http://localhost:4200`.

ANEXO III: Manual Técnico Avanzado de Angular CLI

Dominio de Angular CLI y Automatización de Procesos

1 El Ecosistema de Angular CLI: Fundamentos y Arquitectura

La interfaz de línea de comandos de Angular (CLI) es el motor de orquestación que asegura la integridad de las aplicaciones empresariales. Basada en el motor de *Schematics*, permite realizar transformaciones complejas en el sistema de archivos de manera atómica y segura.

- **Estandarización:** Al utilizar la CLI, se garantiza que cada archivo se ubique en el directorio correcto y que las clases sigan la nomenclatura oficial de Google (*kebab-case* para archivos y *PascalCase* para clases), facilitando la colaboración en equipos multidisciplinares.
- **Productividad mediante la Automatización:** La CLI automatiza tareas repetitivas como la minificación de archivos, la transpilación de TypeScript y la gestión de variables de entorno según el perfil de ejecución (desarrollo, pruebas o producción).

2 Comandos Fundamentales de Gestión de Proyectos

2.1 Inicialización de Proyectos (ng new)

Permite configurar el entorno de trabajo inicial, incluyendo la gestión de paquetes y el motor de estilos. Es el punto de partida que configura el archivo `angular.json`.

```
# Inicializacion con optimizacion empresarial
ng new mi-app-core --style scss --routing --standalone --strict
```

Desglose de Flags Críticos:

- `--style`: Define el preprocesador de CSS (SCSS es el estándar por su capacidad de anidamiento).
- `--routing`: Genera automáticamente el módulo de rutas para la navegación.
- `--standalone`: Crea la aplicación siguiendo la arquitectura moderna sin NgModules.
- `--strict`: Activa reglas de tipado rigurosas para prevenir errores en tiempo de compilación.

2.2 Servidor de Desarrollo (ng serve)

Levanta el entorno local con capacidades de recarga en caliente (*Hot Reload*) y optimización de memoria de ejecución.

```
# Ejecucion estandar y personalizada
ng serve -o --port 5000
```

3 Generación de Elementos (Scaffolding) y Jerarquía Profunda

El comando `generate` (o su alias `g`) es vital para inyectar lógica sin errores manuales. Registra automáticamente las dependencias y configura los metadatos necesarios en la arquitectura del proyecto.

Cuadro 2: Matriz Extendida de Comandos de Scaffolding

Acción	Comando Completo	Versión Corta
Componente	<code>ng generate component</code>	<code>ng g c</code>
Servicio	<code>ng generate service</code>	<code>ng g s</code>
Interfaz	<code>ng generate interface</code>	<code>ng g i</code>
Guardia	<code>ng generate guard</code>	<code>ng g g</code>
Pipe (Filtro)	<code>ng generate pipe</code>	<code>ng g p</code>
Interceptor	<code>ng generate interceptor</code>	<code>ng g interceptor</code>
Directiva	<code>ng generate directive</code>	<code>ng g d</code>

3.1 Componentes (ng g c)

Es la unidad mínima de la interfaz de usuario. Admite banderas de generación avanzadas como `--change-detection OnPush` para optimizar el ciclo de renderizado y `--view-encapsulation` para aislar las reglas CSS.

3.2 Servicios (ng g s)

Contenedores lógicos de negocio decorados con `@Injectable`. Utilizan por defecto la propiedad `providedIn: 'root'` para comportarse como Singletons a nivel global.

3.3 Guards o Guardias (ng g g)

Se utilizan para proteger accesos en rutas del sistema. La CLI actual genera Guards Funcionales, que son más ligeros, eficientes y fáciles de testear que los basados en clases tradicionales.

3.4 Pipes o Tuberías (ng g p)

Transforman datos directamente en las plantillas HTML de manera eficiente. Por defecto son contruidos como *Pure Pipes* para evitar cálculos redundantes.

3.5 Interceptors y Directivas

Los interceptores manipulan peticiones y respuestas HTTP globalmente, mientras que las directivas (`ng g d`) permiten inyectar comportamientos personalizados directamente sobre el DOM.

4 Comandos de Mantenimiento y Calidad

- **Pruebas de Calidad (ng test y ng lint):** Permiten ejecutar suites de pruebas unitarias y realizar análisis estático de código para garantizar la robustez del proyecto.
- **Construcción para Producción (ng build):** Genera el paquete distribuible final optimizado mediante técnicas de *Uglification*, *Minification* y sustitución automática de variables de entorno.

5 Glosario de Términos Técnicos de CLI

- **Functional Guards:** Lógica ligera de seguridad en enrutamiento implementada mediante funciones inyectables.
- **Tree Shaking:** Proceso automático de optimización que elimina el código muerto o no utilizado durante la fase de compilación.
- **Pure Pipes:** Filtros de transformación visual que solo vuelven a calcular su valor si cambian sus argumentos de entrada de forma explícita.
- **Dry Run:** Flag (`-d` o `--dry-run`) que simula la ejecución de un comando de scaffolding mostrando el resultado esperado sin modificar el disco físico.

6 Ejercicios

Implementación de Seguridad Perimetral

Enunciado

Se requiere proteger las rutas de administración y asegurar que cada petición HTTP incluya un token de autorización JWT mediante herramientas generadas por CLI en el subdirectorio de seguridad dedicado.

Solución

```
ng g g p6-ej1/guards/admin-auth --type canActivate
ng g interceptor p6-ej1/interceptors/jwt-token
```

Explicación

Al ejecutar estos comandos con prefijo de carpeta, Angular CLI crea los archivos base necesarios sin colisión de nombres. El Guard funcional permite centralizar la lógica de sesión retornando un booleano que valida el acceso a la ruta. El Interceptor genera la estructura para interceptar la petición (`HttpRequest`), clonarla e inyectar la cabecera `Authorization` antes de transmitirla al servidor.

Transformación y Visualización de Datos

Enunciado

Se solicita que las fechas se muestren en formato relativo (ej. "hace 2 días") y que los importes negativos se resalten en rojo automáticamente mediante artefactos desacoplados generados por CLI.

Solución

```
ng g p p6-ej2/pipes/relative-time
ng g d p6-ej2/directives/negative-highlight
```

Explicación

El Pipe (`ng g p`) registra una clase que implementa `PipeTransform`, permitiendo su uso inmediato en el

HTML con el operador pipe (|). La Directiva (ng g d) inyecta el ElementRef del host, permitiendo aplicar lógica de estilos (style.color) basada en el valor numérico del elemento sin acoplar código a la vista.

Sistema Gestor Pro

Enunciado

Construye la arquitectura de un gestor de activos modular y escalable mediante Angular CLI siguiendo las cuatro fases operativas de ingeniería.

Solución

Fase 1: Infraestructura. Inicializar proyecto con SCSS, enrutamiento y tipado estricto:

```
ng new GestorPro --style scss --routing --strict
```

Fase 2: Modelado. Creación de interfaces y servicio core de gestión de datos:

```
ng g i core/models/activo  
ng g s core/services/activo-data
```

Fase 3: Interfaz. Componente de lista optimizado usando la estrategia de detección OnPush:

```
ng g c features/activo-list --change-detection OnPush --standalone
```

Fase 4: Seguridad y Build. Configurar restricción de seguridad en vistas y compilar para producción:

```
ng g g core/guards/gestor-auth --type canActivate  
ng build --configuration production
```

Explicación

La secuencia de comandos orquesta la creación íntegra de la solución respetando la convención de nombres de Google, configurando la inyección como Singleton para el servicio, optimizando el rendimiento con la estrategia OnPush y generando el paquete final minificado y optimizado con *Tree-Shaking*.

7 Referencias didácticas

Para profundizar en el dominio técnico y la automatización avanzada con Angular CLI, se recomiendan los siguientes recursos oficiales y didácticos:

- **W3Schools Angular Get Started** (https://www.w3schools.com/angular/angular_getstarted.asp): Tutorial guiado paso a paso para la comprensión de los fundamentos y primer contacto con el ecosistema.
- **DesarrolloWeb - Manual de Angular CLI** (<https://desarrolloweb.com/articulos/angular-cli.html>): Monográfico en español que detalla la configuración práctica del entorno de desarrollo y comandos operativos.
- **Angular CLI Official Documentation** (<https://v17.angular.io/cli#installing-angular-cli>): Documentación técnica de referencia mantenida por el equipo de Google con la especificación completa de esquemáticos, modificadores y flags de compilación.

ANEXO IV: Guía Básica de HTML

Fundamentos del Desarrollo Web Estructurado

1 Introducción al Desarrollo Web: ¿Qué es HTML?

HTML, siglas de *HyperText Markup Language* (Lenguaje de Marcado de Hipertexto), es el estándar global indiscutible para la creación de páginas web. No debe confundirse con un lenguaje de programación, ya que no maneja lógica condicional, bucles ni operaciones matemáticas complejas. En su lugar, es un lenguaje de marcado que utiliza una estructura de etiquetas para definir la arquitectura, el esqueleto y el contenido semántico de un documento en la red.

Para entender el ecosistema web moderno, solemos utilizar la analogía del cuerpo humano o la construcción de un edificio:

- **HTML (El Esqueleto):** Define la estructura ósea, las habitaciones, los muros de carga y la distribución del contenido (dónde va un título, un párrafo o una imagen).
- **CSS (La Piel y Vestimenta):** Define el diseño visual, los colores, la tipografía, la disposición espacial y la estética general del sitio (ver ANEXO V).
- **JavaScript (Los Músculos y el Cerebro):** Aporta interactividad, dinamismo, lógica de negocio en cliente y comportamiento reactivo a los estímulos del usuario (ver ANEXO VI).

A continuación se presenta una minipágina que ilustra cómo interactúan HTML, CSS y JavaScript dentro de un mismo archivo, así como la forma en que se referenciarían si estuvieran externalizados:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Portal Corporativo - Demostracion</title>
  <!-- Enlace a hoja de estilo externa: <link rel="stylesheet" href="estilos.css"> -->
  <style>
    body { font-family: sans-serif; background-color: #F8FBFE; padding: 20px; }
    .card { background: white; border: 1px solid #CBD5E1; padding: 15px; border-radius: 8px; }
    .btn { background: #1E75B8; color: white; border: none; padding: 8px 12px; border-radius: 4px; }
  </style>
</head>
<body>
  <div class="card">
    <h1>Portal de Servicios</h1>
    <p id="mensaje">Presione el boton para interactuar con la logica en cliente.</p>
    <button class="btn" onclick="mostrarConfirmacion()">Pulsar Accion</button>
  </div>
  <!-- Enlace a script JavaScript externo: <script src="app.js"></script> -->
  <script>
    function mostrarConfirmacion() {
      document.getElementById('mensaje').textContent = 'Accion registrada exitosamente en cliente.';
    }
  </script>
</body>
</html>
```

Listing 7: p1-intro/minipagina-integrada.html

2 Anatomía de una Etiqueta HTML

El bloque de construcción fundamental de HTML es el elemento. La gran mayoría de los elementos constan de una etiqueta de apertura, el contenido propiamente dicho y una etiqueta de cierre.

2.1 Estructura de un Elemento Estándar

Un elemento HTML se compone de:

- **Etiqueta de apertura (*Opening tag*):** Indica dónde comienza el efecto o estructura (por ejemplo, `<p>`).
- **Atributos (*Attributes*):** Propiedades opcionales que se declaran en la etiqueta de apertura (por ejemplo, `class="..."` o `id="..."`).
- **Contenido (*Content*):** Texto, imagen o elementos anidados dentro de las etiquetas.
- **Etiqueta de cierre (*Closing tag*):** Idéntica a la de apertura pero precedida por una barra inclinada (`</p>`).

```
<p class="destacado">Este es un parrafo de texto didactico.</p>
```

Listing 8: p2-anatomia/ejemplo-etiqueta.html

2.2 Elementos Vacíos o Autoconclusivos

No todos los elementos requieren una etiqueta de cierre. Los elementos vacíos (*void elements*) no albergan texto directo, sino que insertan componentes autónomos, recursos externos o separadores estructurales en el flujo del documento:

```
<!-- Insercion de salto de linea, linea divisoria, imagen e input de formulario -->
<p>Primera linea de texto descriptivo.<br>Segunda linea tras salto forzado.</p>
<hr>

<input type="text" placeholder="Ingrese codigo de acceso" name="codigo">
```

Listing 9: p2-anatomia/elementos-vacios.html

3 La Estructura Global y Obligatoria de un Documento

Cualquier archivo HTML válido debe seguir un esquema jerárquico estricto para garantizar el renderizado correcto y la indexación en motores de búsqueda (SEO).

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Portal Corporativo</title>
</head>
<body>
  <h1>¡Bienvenidos a la plataforma web!</h1>
  <p>Este es el cuerpo visible del documento.</p>
</body>
</html>
```

Listing 10: p3-estructura/plantilla-universal.html

3.1 Desglose de la Plantilla Base

- `<!DOCTYPE html>`: Declaración inicial que comunica al navegador el estándar HTML5.

- `<html lang=.es>`: Contenedor raíz con especificación del idioma primario.
- `<head>`: Cabecera técnica invisible con metadatos, codificación, scripts y títulos.
- `<meta charset=ÜTF-8>`: Codificación universal que soporta caracteres especiales y tildes.
- `<body>`: Cuerpo visible que aloja todo el contenido visual interactivo.

4 Elementos Esenciales para la Estructura de Contenidos

HTML proporciona etiquetas específicas según el rol semántico y tipológico de los datos:

4.1 Encabezados o Títulos (h1 a h6)

Estructuran jerárquicamente los bloques de texto. `<h1>` es el nivel supremo (único por documento en buenas prácticas de SEO), decreciendo en jerarquía hasta `<h6>`.

4.2 Párrafos y Estilos de Texto Comunes

- `<p>`: Bloque de texto con margen vertical automático.
- `<strong>`: Énfasis semántico de gran importancia y representación en negrita.
- `<em>`: Énfasis prosódico y representación visual en cursiva.

4.3 Hipervínculos e Imágenes

```
<a href="https://www.w3.org">Consortio W3C</a>

```

Miniejercicio: Creación de Ficha Descriptiva con Enlaces

Enunciado: Elabora un fragmento HTML que contenga un encabezado de segundo nivel, un párrafo con una palabra clave en negrita y un enlace accesible con destino a un recurso web.

Solución:

```
<h2>Servicio de Monitorizacion Cloud</h2>
<p>El sistema se encuentra en estado <strong>operativo</strong> tras la revision matutina.</p>
<a href="https://estado.servicios-cloud.org">Consultar Registro de Estado</a>
```

Listing 11: p4-elementos/ficha-descriptiva.html

Explicación: Se usa `<h2>` para no duplicar el `<h1>` principal, `<strong>` para aportar semántica de estado crítico y `<a>` con su atributo mandatorio `href`.

5 HTML Semántico: Estructuración Moderna

Las etiquetas semánticas de HTML5 comunican a los motores de indexación y herramientas de accesibilidad el propósito exacto de cada sección:

- `<header>`: Cabecera de página o sección (logotipos, títulos, herramientas).
- `<nav>`: Contenedor exclusivo de enlaces de navegación principal.
- `<main>`: Núcleo de contenido único y exclusivo del documento actual.

- `<article>`: Bloque de información autocontenido y redistribuible de forma autónoma.
- `<section>`: Agrupación temática de contenidos relacionados.
- `<footer>`: Pie de página (derechos de autor, avisos legales y contacto).

Miniejercicio: Estructura Semántica de un Comunicado

Enunciado: Modela un comunicado técnico empleando las etiquetas semánticas `<article>`, `<header>` y `<footer>`.

Solución:

```
<article>
  <header>
    <h3>Lanzamiento de Version v19.2</h3>
    <p>Publicado por el Equipo de Arquitectura de Software</p>
  </header>
  <p>La nueva version incorpora parches criticos en el pipeline de empaquetado.</p>
  <footer>
    <small>Fecha de difusion: 2026-09-20</small>
  </footer>
</article>
```

Listing 12: p5-semantica/comunicado-semantico.html

Explicación: `<article>` garantiza la autonomía del fragmento, mientras que los subbloques `<header>` y `<footer>` aíslan los metadatos de emisión sin recurrir a elementos genéricos `<div>`.

6 Ejercicio final HTML

Portal de Noticias Técnicas Semántico

Enunciado

Construye un documento HTML5 completo y semánticamente estructurado para un portal de novedades que integre:

1. Cabecera con título del sitio y barra de navegación accesible.
2. Área principal (`<main>`) que contenga un artículo (`<article>`) con título, imagen ilustrativa (con texto alternativo), párrafo con énfasis y un elemento autoconclusivo de corte (`<hr>`).
3. Pie de página corporativo con información de derechos.

Solución

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Boletin de Ingenieria Web</title>
  </head>
```

```
<body>
  <header>
    <h1>Boletin de Arquitectura de Software</h1>
    <nav>
      <ul>
        <li><a href="#articulos">Articulos</a></li>
        <li><a href="#recursos">Recursos</a></li>
        <li><a href="#contacto">Contacto</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <article id="articulos">
      <header>
        <h2>Adopcion de Signals en Ecosistemas Frontend</h2>
        <p>Por <em>Comite de Estandares Tecnologicos</em></p>
      </header>
      
      <p>
        La transicion hacia modelos de <strong>reactividad granular</strong> permite suprimir
        la necesidad de barridos recursivos en el arbol del DOM.
      </p>
      <hr>
      <p>Para mayor detalle operativo, consulta la documentacion tecnica.</p>
    </article>
  </main>

  <footer>
    <p><small>&copy; 2026 Plataforma de Ingenieria. Todos los derechos reservados.</small></p>
  </footer>
</body>
</html>
```

Listing 13: p6-ejfinal/portal-noticias.html

Explicación

- **Estructura semántica jerarquizada:** El documento arranca con la declaración formal `<!DOCTYPE html>` e idioma en `<html>`. La jerarquía textual respeta el uso único de `<h1>` en la cabecera general y desciende a `<h2>` dentro del contenedor autónomo `<article>`.
- **Accesibilidad y buenas prácticas:** La imagen incorpora el atributo descriptivo `alt`, la navegación se estructura con listas desordenadas (`<ul>`) dentro de `<nav>` y se integran elementos de formato enfático (`<strong>`, `<em>`) y autoconclusivos (`<hr>`).

7 Referencias didácticas

Para profundizar en las especificaciones del consorcio web y estándares de marcado HTML:

- **W3Schools HTML Tutorial** (<https://www.w3schools.com/html/default.asp>): Manual de referencia práctico con editores en vivo para ensayar etiquetas, atributos y formularios interactivos.
- **MDN Web Docs - Sintaxis Básica de HTML** (https://developer.mozilla.org/es/docs/Learn_web_

[development/Core/Structuring_content/Basic_HTML_syntax](#)): Guía técnica rigurosa de Mozilla sobre la anatomía de elementos, atributos booleanos y anidamiento correcto.

- **Microsoft Learn - Creación de páginas web con HTML y CSS** (<https://learn.microsoft.com/es-es/training/paths/build-web-pages-html-css-for-beginners/>): Ruta de aprendizaje oficial con laboratorios prácticos orientados a la accesibilidad y estructuración profesional.

ANEXO V: Guía Básica de CSS

Conceptos Fundamentales para el Diseño y Maquetación Web

1 Introducción a CSS: El Alma Visual de la Web

El lenguaje HTML define de forma exclusiva la estructura de una página web, determinando qué elementos componen el contenido (títulos, párrafos, imágenes o tablas). Sin embargo, carece de herramientas nativas optimizadas para gestionar la apariencia visual. Es aquí donde entra en juego CSS (*Cascading Style Sheets* o Hojas de Estilo en Cascada), el estándar desarrollado por el W3C para separar la estructura semántica de la presentación estética.

CSS permite controlar de manera centralizada aspectos como los colores, los tipos de letra, la disposición de los bloques, los tamaños de las cajas y la adaptabilidad a dispositivos móviles. Cuando existen varias propiedades contradictorias para un mismo elemento HTML, CSS las resuelve basándose en tres factores clave: la importancia de la regla, la especificidad del selector y el orden de declaración.

2 Sintaxis Fundamental de una Regla CSS

Una declaración CSS estándar consta de tres elementos esenciales:

- **Selector:** Identifica el elemento o elementos HTML destino.
- **Propiedad:** Representa la característica visual que se desea modificar (ej. `color`, `font-size`).
- **Valor:** Especifica la magnitud o configuración exacta asignada a la propiedad.

```
/* Selector que apunta a todos los parrafos de la pagina */
p {
  color: #2c3e50;      /* Cambia el color del texto */
  font-size: 16px;     /* Define el tamaño de la fuente */
  line-height: 1.6;    /* Ajusta la altura de la línea */
}
```

Listing 14: p2-sintaxis/ejemplo-regla.css

Miniejercicio: Declaración de Regla para Títulos

Enunciado: Escribe una regla CSS que configure todos los encabezados `<h2>` con color azul marino (`#1F2758`) y una alineación centrada.

Solución:

```
h2 {
  color: #1F2758;
  text-align: center;
}
```

Listing 15: p2-sintaxis/regla-h2.css

Explicación: El selector es `h2`, asignando `#1F2758` a la propiedad `color` y el valor `center` a la propiedad de alineación `text-align`.

3 Métodos de Integración de CSS en HTML

Existen tres formas básicas de integrar hojas de estilo CSS dentro de un documento HTML:

3.1 CSS Externo (Recomendado)

Archivo independiente con extensión .css vinculado a través de la etiqueta `<link>` en el `<head>`. Facilita el mantenimiento global y aprovecha la caché del navegador:

```
<head>
  <link rel="stylesheet" href="estilos.css">
</head>
```

Listing 16: p3-metodos/vinculacion-externa.html

3.2 CSS Interno

Reglas delimitadas por la etiqueta contenedora `<style>` dentro del `<head>`:

```
<head>
  <style>
    body { background-color: #f4f6f7; }
  </style>
</head>
```

Listing 17: p3-metodos/estilo-interno.html

3.3 CSS en Línea (Inline)

Añadido mediante el atributo global `style` en la etiqueta de apertura de un elemento. Desaconsejado en producción por romper la modularidad:

```
<p style="color: #e74c3c; font-weight: bold;">Texto resaltado en linea.</p>
```

Listing 18: p3-metodos/estilo-inline.html

Miniejercicio: Integración de Hoja Externa

Enunciado: Desarrolla la sección `<head>` de un documento HTML enlazando un archivo externo llamado `layout.css` y definiendo un color de fondo para la clase `.panel` mediante un bloque `<style>` interno de respaldo.

Solución:

```
<head>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="layout.css">
  <style>
    .panel { background-color: #F8FBFE; }
  </style>
</head>
```

Listing 19: p3-metodos/integracion-mixta.html

Explicación: La etiqueta `<link>` delega las reglas principales al archivo externo mientras que `<style>` permite sobrescribir estilos locales para esa vista específica.

4 Los Selectores Básicos más Utilizados

Los tres selectores esenciales para formatear partes específicas de la interfaz son:

- **Selector de Elemento:** Aplica estilos a todas las instancias de una etiqueta HTML (ej. `h2`).
- **Selector de Clase (.):** Agrupar elementos bajo un identificador estético reutilizable anteponiendo un punto al

nombre (ej. `.boton-primario`).

- **Selector de ID (#):** Selecciona de forma unívoca un único elemento anteponiendo una almohadilla al identificador (ej. `#cabecera-principal`).

```
/* Selector de Elemento */
h2 { color: #2980b9; }

/* Selector de Clase */
.boton-primario {
  background-color: #2ecc71;
  border-radius: 4px;
}

/* Selector de ID */
#cabecera-principal {
  padding: 20px;
  border-bottom: 2px solid #ddd;
}
```

Listing 20: p4-selectores/selectores-practicos.css

Miniejercicio: Jerarquía de Selectores

Enunciado: Define una clase `.alerta` con fondo ámbar suave y un ID `#alerta-critica` que fuerce el texto en negrita y color rojo.

Solución:

```
.alerta {
  background-color: #FEF3C7;
  padding: 12px;
}
#alerta-critica {
  color: #DC2626;
  font-weight: bold;
}
```

Listing 21: p4-selectores/selectores-alerta.css

Explicación: La clase puede aplicarse a múltiples avisos, pero el selector de ID posee mayor especificidad y aplica un formato prioritario sobre el aviso concreto.

5 El Modelo de Caja (Box Model)

En la arquitectura de un navegador, cada elemento HTML se modela visualmente como una caja rectangular que ocupa un espacio en pantalla. Consta de cuatro áreas concéntricas:

1. **Content (Contenido):** Área central donde reside el texto o la imagen, dimensionada con `width` y `height`.
2. **Padding (Relleno):** Espacio interior transparente entre el contenido y el borde.
3. **Border (Borde):** Línea delimitadora a la que se le asigna grosor, estilo y color.
4. **Margin (Margen):** Espacio exterior transparente que separa la caja de sus vecinas.

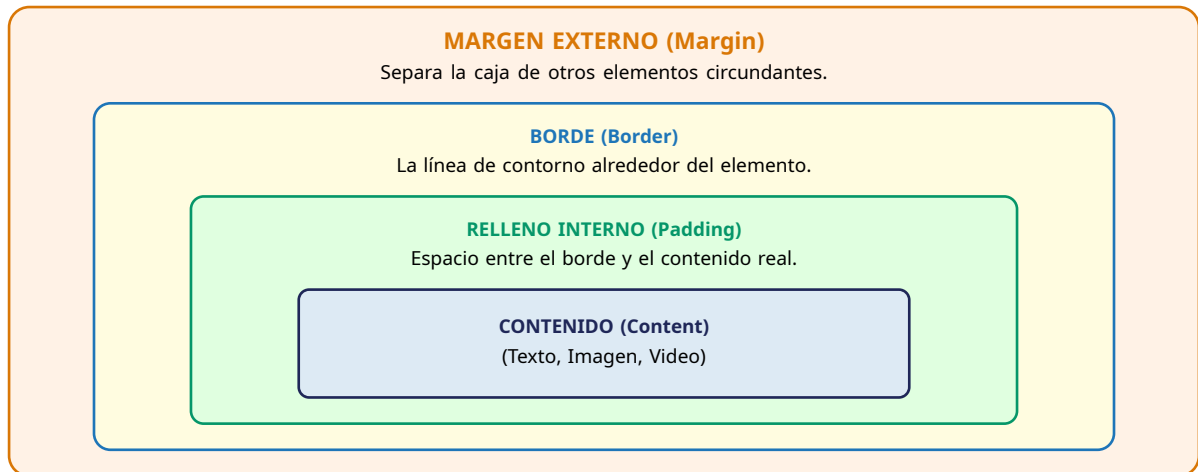


Figura 1: Representación conceptual del Modelo de Caja de CSS.

Miniejercicio: Configuración del Modelo de Caja

Enunciado: Crea una tarjeta de notificación con ancho de 250px, relleno interno de 15px, borde sólido azul de 2px y margen inferior de 20px.

Solución:

```
.caja-notificacion {  
  width: 250px;  
  padding: 15px;  
  border: 2px solid #1E75B8;  
  margin-bottom: 20px;  
}
```

Listing 22: p5-boxmodel/caja-notificacion.css

Explicación: Las cuatro propiedades configuran de forma secuencial las capas del modelo: dimensión del contenido, colchón interior, marco exterior y distanciamiento vertical.

6 Propiedades Esenciales de Diseño

Las propiedades fundamentales para estructurar páginas web se clasifican en:

6.1 Tipografía y Texto

- `font-family`: Familia tipográfica a renderizar.
- `font-size`: Dimensión tipográfica (px, rem, %).
- `text-align`: Alineación horizontal (left, center, right, justify).

6.2 Colores y Fondos

- `color`: Color del texto (nombres, hexadecimal #FFFFFF, RGB/RGBA).
- `background-color`: Tonalidad de fondo de la caja del elemento HTML.

6.3 Dimensiones

- `width / height`: Anchura y altura absoluta o porcentual del contenedor.

Miniejercicio: Botón de Acción Estilizado

Enunciado: Combina propiedades tipográficas, cromáticas y de dimensiones para estilizar un botón interactivo.

Solución:

```
.btn-accion {
  font-family: 'Segoe UI', sans-serif;
  font-size: 14px;
  color: #FFFFFF;
  background-color: #059669;
  width: 160px;
  text-align: center;
  padding: 10px;
  border-radius: 6px;
}
```

Listing 23: p6-propiedades/boton-estilo.css

Explicación: Se asegura legibilidad con tipografía sin serifa, contraste cromático accesible (texto blanco sobre esmeralda) y centrado de texto dentro de una dimensión fija.

7 Ejercicio final CSS

Maquetación Profesional de Tarjeta de Perfil

Enunciado

Diseña y maqueta la hoja de estilos de una tarjeta de perfil de usuario profesional que cumpla los siguientes requerimientos:

1. Contenedor de ancho fijo (300px), centrado horizontalmente en la pantalla, con fondo blanco, bordes redondeados y sombra sutil de elevación.
2. Título con tipografía diferenciada en tono corporativo azul y margen inferior controlado.
3. Párrafo descriptivo con tipografía neutra, tamaño reducido e interlineado óptimo para lectura.
4. Botón de contacto que responda al modelo de caja con relleno simétrico y color de fondo verde esmeralda.

Solución

```
/* 1. Contenedor principal de la tarjeta */
.tarjeta-perfil {
  width: 300px;
  background-color: #FFFFFF;
  border: 1px solid #E2E8F0;
  border-radius: 10px;
  padding: 24px;
```

```
margin: 30px auto;
text-align: center;
box-shadow: 0 4px 6px rgba(0, 0, 0, 0.08);
}

/* 2. Titulo del perfil */
.tarjeta-perfil h3 {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  color: #1F2758;
  font-size: 20px;
  margin-top: 0;
  margin-bottom: 8px;
}

/* 3. Descripcion profesional */
.tarjeta-perfil p {
  color: #64748B;
  font-size: 13pt;
  line-height: 1.5;
  margin-bottom: 20px;
}

/* 4. Boton interactivo */
.tarjeta-perfil .btn-perfil {
  display: inline-block;
  background-color: #059669;
  color: #FFFFFF;
  text-decoration: none;
  font-size: 12pt;
  font-weight: bold;
  padding: 10px 18px;
  border-radius: 6px;
  transition: background-color 0.2s ease;
}
```

Listing 24: p7-ejfinal/tarjeta-perfil.css

Explicación

- **Centrado y Modelo de Caja:** El valor `margin: 30px auto` aprovecha los márgenes automáticos izquierdo y derecho para centrar la caja de 300px en el viewport horizontal. El relleno `padding: 24px` crea el aislamiento perimetral indispensable para que el texto respire.
- **Jerarquía y contraste visual:** El título utiliza el color institucional #1F2758 para enfatizar la identidad del perfil, mientras que el párrafo secundario utiliza un gris atenuado (#64748B) y el botón un acento esmeralda (#059669) que actúa como llamada a la acción (*Call to Action*).

8 Referencias didácticas

Para profundizar en las especificaciones de maquetación moderna, flexbox y diseño adaptable con CSS:

- **W3Schools CSS Tutorial** (<https://www.w3schools.com/css/default.asp>): Compendio de referencia interactivo con ejemplos ejecutables de selectores, animaciones y diseño responsivo.
- **MDN Web Docs - Estilizando el Contenido** (https://developer.mozilla.org/es/docs/Learn_web_

[development/Getting_started/Your_first_website/Styling_the_content](#)): Guía fundamental de Mozilla que analiza en profundidad la mecánica de cascada, especificidad e integración de fuentes tipográficas.

- **Microsoft Learn - Uso de estilos CSS en páginas web** (<https://learn.microsoft.com/es-es/training/modules/use-css-styles/>): Módulo de aprendizaje profesional enfocado en el modelo de caja, posicionamiento espacial y buenas prácticas de maquetación limpia.

ANEXO VI: Guía Básica de JavaScript

Fundamentos de Dinamismo, Manipulación del DOM y Lógica en Cliente

1 Introducción a JavaScript: El Cerebro de la Web

JavaScript es el lenguaje de programación interpretado de alto nivel que dota a las páginas web de interactividad y comportamiento reactivo en tiempo de ejecución. Mientras que HTML provee la estructura (ver ANEXO IV) y CSS define el aspecto visual (ver ANEXO V), JavaScript gestiona la lógica, escucha los eventos del usuario, modifica el documento en tiempo real y coordina la transferencia de datos con servidores mediante peticiones asíncronas.

Los navegadores modernos ejecutan JavaScript a través de motores ultra-optimizados (como V8 en Chromium o SpiderMonkey en Firefox) que compilan el código fuente justo a tiempo (*Just-In-Time Compilation* - JIT).

1.1 Declaración de Variables en ECMAScript Moderno

En el estándar ECMAScript actual (ES6+), se han descartado las declaraciones ambiguas con `var` en favor de variables con ámbito de bloque:

- `const`: Declara constantes cuyo enlace de referencia en memoria es inmutable. Debe ser la opción por defecto.
- `let`: Declara variables mutables cuyo valor puede reasignarse durante la ejecución dentro de su bloque delimitador.

```
const puertoServicio = 8080;
let usuariosConectados = 14;

// Mutacion permitida sobre variable declarada con let
usuariosConectados = usuariosConectados + 1;
console.log(`Puerto: ${puertoServicio}, Conectados: ${usuariosConectados}`);
```

Listing 25: p1-variables/variables-modernas.js

Miniejercicio: Gestión Inmutable de Constantes

Enunciado: Declara la URL base de una API corporativa como constante inmutable y una variable para el contador de reintentos, mostrando su valor en consola.

Solución:

```
const API_URL = 'https://api.servicios-cloud.org/v1';
let intentosFallidos = 0;

intentosFallidos++;
console.log(`Conectando a ${API_URL} (Intento numero ${intentosFallidos})`);
```

Listing 26: p1-variables/constantes-api.js

Explicación: Se usa `const` en mayúsculas para valores de configuración fijos e inmutables, reservando `let` para contadores que sufren reasignaciones numéricas.

2 Estructuras de Control y Funciones Modernas

JavaScript dispone de estructuras condicionales clásicas (`if/else`, `switch`) y mecanismos iterativos para arreglos como `.map()`, `.filter()` o bucles `for...of`.

2.1 Funciones Flecha (Arrow Functions)

Las funciones flecha proporcionan una sintaxis concisa y mantienen el enlace léxico de la palabra reservada `this`, siendo el estándar en el desarrollo contemporáneo:

```
// Declaracion formal de funcion flecha compacta
const calcularIva = (importe, tasa = 0.21) => importe * tasa;

const total = 100 + calcularIva(100);
console.log(`Total con impuesto: ${total} EUR`);
```

Listing 27: p2-funciones/funciones-flecha.js

Miniejercicio: Filtrado de Métricas con Funciones Flecha

Enunciado: Dado un arreglo de latencias numéricas, filtra aquellas que superen el límite crítico de 100 milisegundos utilizando el método `.filter()`.

Solución:

```
const latencias = [45, 120, 60, 150, 30];
const latenciasCriticas = latencias.filter(ms => ms > 100);

console.log('Nodos sobrecargados:', latenciasCriticas); // [120, 150]
```

Listing 28: p2-funciones/filtrado-metricas.js

Explicación: La función flecha `ms => ms > 100` evalúa cada elemento devolviendo un booleano, permitiendo un procesamiento declarativo y conciso.

3 Manipulación del DOM y Manejo de Eventos

El *Document Object Model* (DOM) es la representación en árbol estructurado que el navegador construye a partir del código HTML. JavaScript puede consultar y mutar cualquier nodo del DOM en caliente:

- `document.querySelector(selector)`: Devuelve el primer elemento que coincida con el selector CSS facilitado.
- `element.addEventListener(evento, callback)`: Asocia un escuchador para capturar interacciones del usuario (como clics, escritura o desplazamientos).

```
// 1. Seleccion de elementos del DOM
const botonRecargar = document.querySelector('#btn-recargar');
const etiquetaEstado = document.querySelector('.texto-estado');

// 2. Vinculacion reactiva de evento click
botonRecargar.addEventListener('click', () => {
  etiquetaEstado.textContent = 'Actualizando registros...';
  etiquetaEstado.style.color = '#1E75B8';
});
```

Listing 29: p3-dom/manipulacion-dom.js

Miniejercicio: Contador Interactivo en DOM

Enunciado: Desarrolla la lógica para que al pulsar un botón con ID `#btn-sumar`, se incremente el valor numérico visible de un elemento `<span>` con ID `#valor-contador`.

Solución:

```
const btn = document.querySelector('#btn-sumar');
const display = document.querySelector('#valor-contador');
let cuenta = 0;

btn.addEventListener('click', () => {
  cuenta++;
  display.textContent = cuenta;
});
```

Listing 30: p3-dom/contador-dom.js

Explicación: Se mantiene el estado en una variable local (cuenta) y se actualiza la propiedad textual nativa `.textContent` del nodo DOM en cada evento.

4 Asincronía: Promesas y Fetch API

La naturaleza monohilo de JavaScript requiere que las operaciones intensivas (como solicitudes HTTP o consultas a bases de datos) se ejecuten de manera no bloqueante. Las palabras reservadas `async` y `await` permiten redactar flujos asíncronos con apariencia secuencial limpia:

```
async function consultarDatosServicio(idServicio) {
  try {
    const respuesta = await fetch(`https://api.servicios-cloud.org/nodos/${idServicio}`);
    if (!respuesta.ok) {
      throw new Error(`Error en comunicacion HTTP: ${respuesta.status}`);
    }
    const datos = await respuesta.json();
    console.log('Datos procesados:', datos);
  } catch (error) {
    console.error('Fallo en la sincronizacion:', error.message);
  }
}
```

Listing 31: p4-async/peticion-asincrona.js

Miniejercicio: Consumo Asíncrono de un Recurso JSON

Enunciado: Define una función asíncrona que consulte una URL pública y retorne el número de elementos obtenidos del JSON remoto.

Solución:

```
async function contarUsuariosRemotos() {
  const res = await fetch('https://jsonplaceholder.typicode.com/users');
  const usuarios = await res.json();
  return usuarios.length;
}
```

Listing 32: p4-async/consumo-json.js

Explicación: La directiva `await` pausa la ejecución interna de la función hasta que se resuelva la Promesa HTTP y la posterior serialización a objeto JavaScript.

5 Ejercicio final JavaScript

Monitor Interactivo Asíncrono con Mutación de Interfaz

Enunciado

Desarrolla una solución completa en JavaScript para una página web que cumpla los siguientes requerimientos técnicos:

1. Escucha el evento de envío (*submit*) de un formulario con ID `#form-nodo`.
2. Previén la recarga nativa de la página mediante `event.preventDefault()`.
3. Extrae el identificador ingresado en un campo de texto con ID `#input-id`.
4. Realiza una consulta asíncrona mediante `fetch` a un servicio remoto simulado.
5. Muestra un mensaje de carga transitorio y renderiza el resultado obtenido dentro de un contenedor con ID `#resultado-nodo`, manejando posibles excepciones mediante bloques `try/catch`.

Solución

```
// 1. Vinculacion a elementos del arbol DOM
const formulario = document.querySelector('#form-nodo');
const inputId = document.querySelector('#input-id');
const contenedorResultado = document.querySelector('#resultado-nodo');

// 2. Gestion del evento de envio del formulario
formulario.addEventListener('submit', async (event) => {
  event.preventDefault(); // Evitamos la recarga total de pagina

  const nodoId = inputId.value.trim();
  if (!nodoId) {
    contenedorResultado.innerHTML = '<p style="color: #DC2626;">Ingrese un identificador valido
    ↪ .</p>';
    return;
  }

  // Indicador de carga activo
  contenedorResultado.innerHTML = '<p style="color: #1E75B8;">Consultando telemetria del nodo
  ↪ ...</p>';

  try {
    const respuesta = await fetch(`https://jsonplaceholder.typicode.com/users/${nodoId}`);
    if (!respuesta.ok) {
      throw new Error(`Nodo inexistente (Estado HTTP: ${respuesta.status})`);
    }
  }

  const datos = await respuesta.json();

  // 3. Renderizado dinamico del nodo procesado
  contenedorResultado.innerHTML = `
    <div style="border: 1px solid #059669; background: #ECFDF5; padding: 12px; border-radius:
    ↪ 6px;">
      <h4 style="margin: 0; color: #065F46;">Nodo Activo: ${datos.name}</h4>
      <p style="margin: 4px 0 0; font-size: 12px; color: #047857;">Email: ${datos.email}</p>
    </div>
```

```
`;  
} catch (error) {  
  contenedorResultado.innerHTML = `  
    <div style="border: 1px solid #DC2626; background: #FEF2F2; padding: 12px; border-radius:  
    ↪ 6px;">  
      <p style="margin: 0; color: #991B1B; font-weight: bold;">Error al consultar telemetria  
      ↪ :</p>  
      <p style="margin: 4px 0 0; font-size: 12px; color: #B91C1C;">${error.message}</p>  
    </div>  
  `;  
}  
});
```

Listing 33: p5-ejfinal/monitor-interactivo.js

Explicación

- **Intercepción de ciclo de vida del formulario:** El método `event.preventDefault()` neutraliza el comportamiento tradicional del navegador de refrescar el documento completo, habilitando una experiencia auténtica de Single Page Application (SPA, ver ANEXO I).
- **Asincronía y actualización atómica del DOM:** La función flecha marcada como `async` permite esperar de forma transparente el resultado de la petición de red con `await fetch()`. La propiedad `.innerHTML` modifica dinámicamente el fragmento visual del contenedor según el estado de la comunicación (carga, éxito o captura de errores en el bloque `catch`).

6 Referencias didácticas

Para profundizar en las capacidades de JavaScript moderno, programación funcional y asincronía avanzada:

- **W3Schools JavaScript Tutorial** (<https://www.w3schools.com/js/default.asp>): Manual didáctico con ejemplos prácticos para la manipulación del DOM, objetos, eventos y métodos de arreglos.
- **MDN Web Docs - JavaScript: Primeros Pasos** (https://developer.mozilla.org/es/docs/Learn_web_development/Core/Scripting): Documentación oficial exhaustiva de Mozilla que cubre desde los tipos de datos primarios hasta promesas, closures y APIs del navegador.
- **Microsoft Learn - Introducción al desarrollo web con JavaScript** (<https://learn.microsoft.com/es-es/training/modules/web-development-101-javascript/>): Ruta de formación técnica estructurada en módulos para comprender la ejecución en cliente, control de flujo y desarrollo de interfaces reactivas.